OPENMP

2026 SPRING

JINHO LEE

ECE, SNU

Accelerated
Intelligent
Systems Lab.

CONTENT

- OpenMP-why?
- Directives
- Critical
- Shared/private
- Single/master
- Sections
- Barriers
- Master
- Nowait
- Reduction
- Scheduling
- Collapse

THREADS

- With threads, you can describe what each thread does
  - With full flexibility

- But do you really need that much flexibility?

THREADS

- With threads, you can describe what each thread does
  - With full flexibility
- But do you really need that flexibility?
- Consider the mac example from last lecture

```
int main()
{
  …

  for(long long int i=0;i<N;i++) {
    c[i] += k[i]*a[i];
    c[i] += k[i]*b[i];
  }

}
  return 0;
}
```

- Each thread performs almost the same job
  - Almost like an extremely long SIMD processing

OPENMP

- With OpenMP, parallelizing a loop is surprisingly simple

```
int main()
{
  …
#pragma omp parallel for
  for(long long int i=0;i<N;i++) {
    c[i] += k[i]*a[i];
    c[i] += k[i]*b[i];
  }

}
  return 0;
}
```

- Compile with
  - g++ -fopenmp mac_omp.cc
- OpenMP automatically parallelizes your code, spawns adequate number of threads and schedules them

OPENMP

- OpenMP is based on “directive”s
  - E.g., `#pragma omp parallel for`

(+) Easy parallelization

(−) Simple syntax

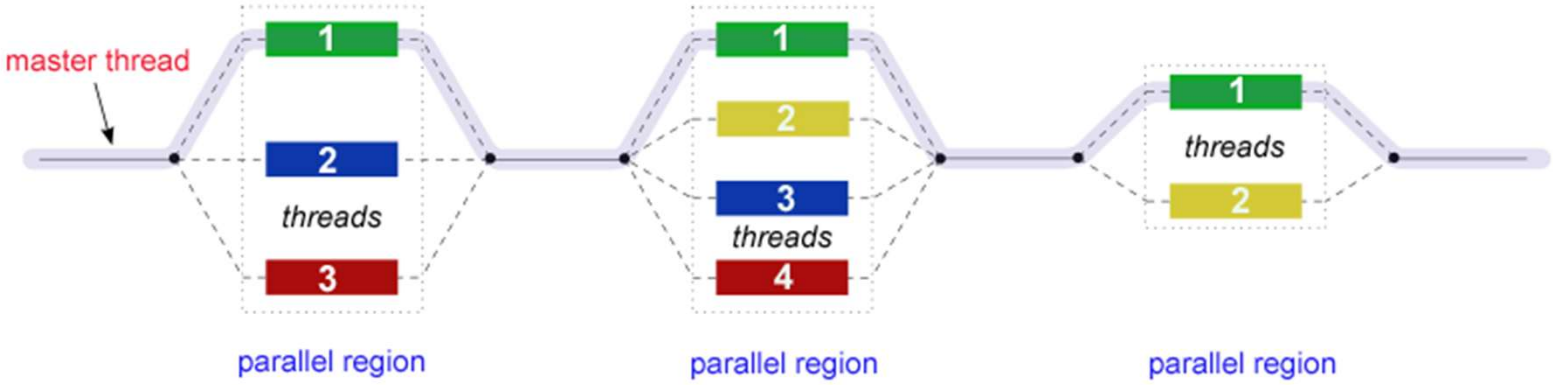
(−) Dangerous

- What the compiler is doing can be uncertain (if you don’t know what you’re doing)

OPENMP DIRECTIVES

- #pragma omp parallel
- { // structured block ... }

- This directive define the parallel region
- Code within {} is parallelized with fork-join
- `#pragma omp parallel for` is used for parallelizing for loops



OPENMP DIRECTIVES

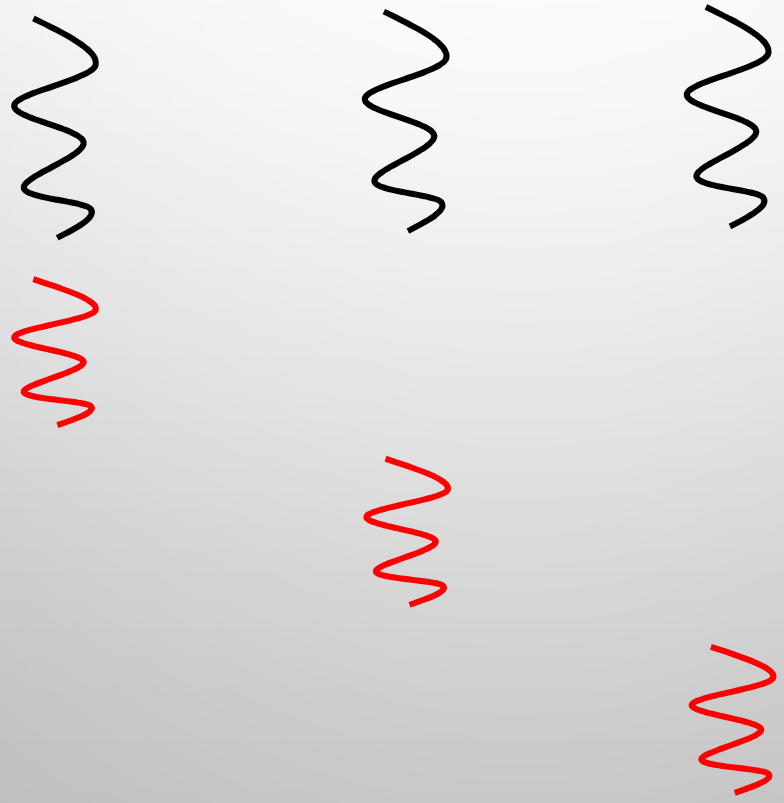
- Gets dirty as you start doing complicated things..
- Critical
- Shared/private
- Single/master
- Sections
- Nowait
- Reduction
- Scheduling
- Collapse
- Others..

CRITICAL SECTION

- Critical section: a code segment that has to be executed one by one

```
m.lock()
//Critical section…
m.unlock()
```

```
#pragma omp critical
{
//Critical section…
}
```

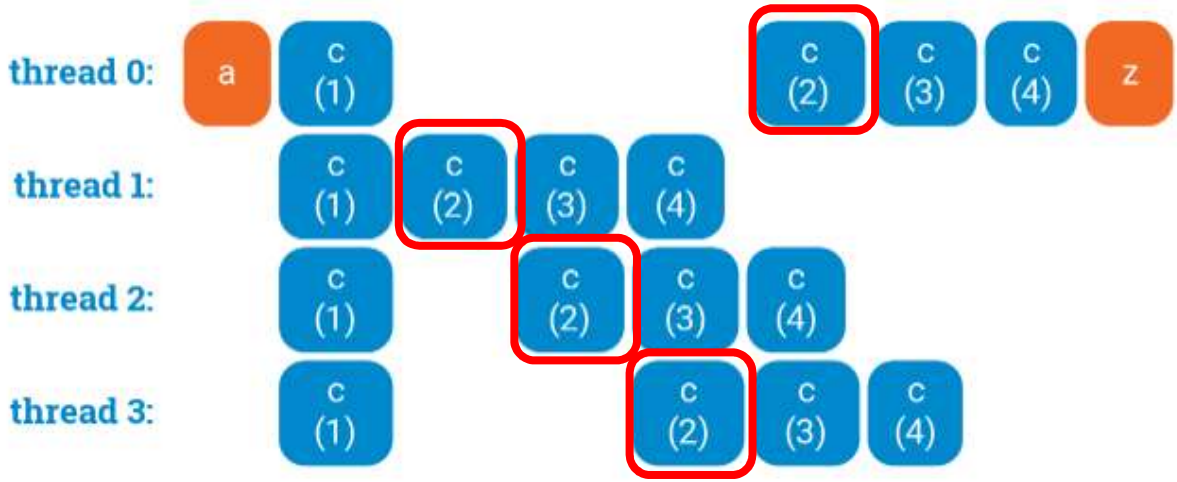


Parallel section

Critical section

OMP PARALLEL+CRITICAL SECTION

```
a();
#pragma omp parallel
{
    c(1);
    #pragma omp critical
    {
        c(2);
    }
    c(3);
    c(4);
}
z();
```



- `omp parallel` defines the region that will be run by all threads

[Image credit to: http://ppc.cs.aalto.fi/ch3/](http://ppc.cs.aalto.fi/ch3/)

SHARED / PRIVATE

- In std::threads, variables inside function being executed is always private
- But for OpenMP?? Is somevar shared or private?

```
int main()
{
  …
  int somevar = 0;
#pragma omp parallel for
  for(long long int i=0;i<N;i++) {
    c[i] += k[i]*a[i];
    c[i] += k[i]*b[i];
    //what happens if somevar accessed here?
  }

}
  return 0;
}
```

- A: it is <u>shared</u> by default.
- Then how do we make it private?

SHARED / PRIVATE

```
#include <omp.h>
#include <cstdio>
#include <cassert>
#include <stdlib.h>

int main(int argc, char **argv)
{

    int shared_int = -1;
    omp_set_num_threads(2); //set the number of threads per parallel block

#pragma omp parallel
    {
        // shared_int is shared
        int tid = omp_get_thread_num();
        printf("Thread ID %2d | shared_int = %d\n", tid, shared_int);
    }
     …
}
```

- If you modify the shared_int, it will result in a race condition

PRIVATE

```
#include <omp.h>
#include <cstdio>
#include <cassert>
#include <stdlib.h>

int main(int argc, char **argv)
{
    omp_set_num_threads(2);

    int is_private = -2;

#pragma omp parallel private(is_private) {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d | is_private(before) = %d\n", tid, is_private);
        is_private = tid;
        printf("Thread ID %2d | is_private(after) = %d\n", tid, is_private);
        assert(is_private == tid);
    }

    printf("Main thread | is_private = %d\n", is_private);

    return 0;
}
```

PRIVATE

```
     int is_private = -2;

#pragma omp parallel private(is_private)
    {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d | is_private(before) = %d\n", tid, is_private);
        is_private = tid;
        printf("Thread ID %2d | is_private(after) = %d\n", tid, is_private);
        assert(is_private == tid);
    }

    printf("Main thread | is_private = %d\n", is_private);
```

```
Thread ID  1 | is_private(before) = 32763
Thread ID  1 | is_private(after) = 1
Thread ID  0 | is_private(before) = 0
Thread ID  0 | is_private(after) = 0
Main thread | is_private = -2
```

- A private *is_private* variable is allocated to each thread
  - Not initialized!
- Modifying *is_private* within parallel block does not modify the value outside the block

FIRSTPRIVATE

```
     int is_private = -2;

#pragma omp parallel firstprivate(is_private)
    {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d | is_private(before) = %d\n", tid, is_private);
        is_private = tid;
        printf("Thread ID %2d | is_private(after) = %d\n", tid, is_private);
        assert(is_private == tid);
    }

    printf("Main thread | is_private = %d\n", is_private);
```

```
Thread ID  1 | is_private(before) = -2
Thread ID  1 | is_private(after) = 1
Thread ID  0 | is_private(before) = -2
Thread ID  0 | is_private(after) = 0
Main thread | is_private = -2
```

- A private *is_private* variable is allocated to each thread
  - ~~Not initialized!~~ Initialized to the value outside the region
- Modifying *is_private* within parallel block does not modify the value outside the block

LASTPRIVATE

```
    int last_private=-1;

#pragma omp parallel for lastprivate(last_private)
    for(int i=0;i<10;i++)
    {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d executing i=%d|
              last_private(before) = %d\n", tid, i, last_private);
        last_private=i;
        printf("Thread ID %2d executing i=%d|
              last_private(after) = %d\n", tid, i, last_private);
        assert(last_private == i);
    }
    printf("Main thread | last_private = %d\n", last_private);
```

- A private *last_private* variable is allocated to each thread
  - Not initialized!
- Last_private becomes what's written in the last iteration (no matter when, which thread executed it!)

LASTPRIVATE

```
    int last_private=-1
#pragma omp parallel fo
    for(int i=0;i<10;i+
    {
        int tid = omp_g
        printf("Thread
              last_priv
        last_private=i;
        printf("Thread
              last_priv
        assert(last_pri
    }
    printf("Main thread
```

```
Thread ID  1 executing i=5| last_private(before) = 1704464448
Thread ID  1 executing i=5| last_private(after) = 5
Thread ID  1 executing i=6| last_private(before) = 5
Thread ID  1 executing i=6| last_private(after) = 6
Thread ID  1 executing i=7| last_private(before) = 6
Thread ID  1 executing i=7| last_private(after) = 7
Thread ID  1 executing i=8| last_private(before) = 7
Thread ID  1 executing i=8| last_private(after) = 8
Thread ID  1 executing i=9| last_private(before) = 8
Thread ID  1 executing i=9| last_private(after) = 9
Thread ID  0 executing i=0| last_private(before) = 0
Thread ID  0 executing i=0| last_private(after) = 0
Thread ID  0 executing i=1| last_private(before) = 0
Thread ID  0 executing i=1| last_private(after) = 1
Thread ID  0 executing i=2| last_private(before) = 1
Thread ID  0 executing i=2| last_private(after) = 2
Thread ID  0 executing i=3| last_private(before) = 2
Thread ID  0 executing i=3| last_private(after) = 3
Thread ID  0 executing i=4| last_private(before) = 3
Thread ID  0 executing i=4| last_private(after) = 4
Main thread | last_private = 9
```

- A private *last_private* variable is allocated to each thread
  - Not initialized!
- Last_private becomes what's written in the last iteration (no matter when, which thread executed it!)

FIRSTPRIVATE, LASTPRIVATE

```
    int last_private=-1;
#pragma omp parallel for firstprivate(last_private) lastprivate(last_private)
for(int i=0;i<10;i++)
    {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d executing i=%d|
              last_private(before) = %d\n", tid, i, last_private);
        last_private=i;
        printf("Thread ID %2d executing i=%d|
              last_private(after) = %d\n", tid, i, last_private);
        assert(last_private == i);
    }
    printf("Main thread | last_private = %d\n", last_private);
```

- A private *last_private* variable is allocated to each thread
  - Initialized to the variable value
- Last_private becomes what's written in the last iteration (no matter when, which thread executed it!)

FIRSTPRIVATE, LASTPRIVATE

```
    int last_private=-1
#pragma omp parallel fo
for(int i=0;i<10;i++)
    {
        int tid = omp_g
        printf("Thread
              last_priv
        last_private=i;
        printf("Thread
              last_priv
        assert(last_pri
    }
    printf("Main thread
```

```
Thread ID  1 executing i=5| last_private(before) = -1
Thread ID  1 executing i=5| last_private(after) = 5
Thread ID  1 executing i=6| last_private(before) = 5
Thread ID  1 executing i=6| last_private(after) = 6
Thread ID  1 executing i=7| last_private(before) = 6
Thread ID  0 executing i=0| last_private(before) = -1
Thread ID  0 executing i=0| last_private(after) = 0
Thread ID  0 executing i=1| last_private(before) = 0
Thread ID  0 executing i=1| last_private(after) = 1
Thread ID  0 executing i=2| last_private(before) = 1
Thread ID  0 executing i=2| last_private(after) = 2
Thread ID  0 executing i=3| last_private(before) = 2
Thread ID  0 executing i=3| last_private(after) = 3
Thread ID  0 executing i=4| last_private(before) = 3
Thread ID  0 executing i=4| last_private(after) = 4
Thread ID  1 executing i=7| last_private(after) = 7
Thread ID  1 executing i=8| last_private(before) = 7
Thread ID  1 executing i=8| last_private(after) = 8
Thread ID  1 executing i=9| last_private(before) = 8
Thread ID  1 executing i=9| last_private(after) = 9
Main thread | last_private = 9
```

- A private *last_priva*
  - Initialized to the variable value

- Last_private becomes what's written in the last iteration (no matter when, which thread executed it!)

SECTIONS

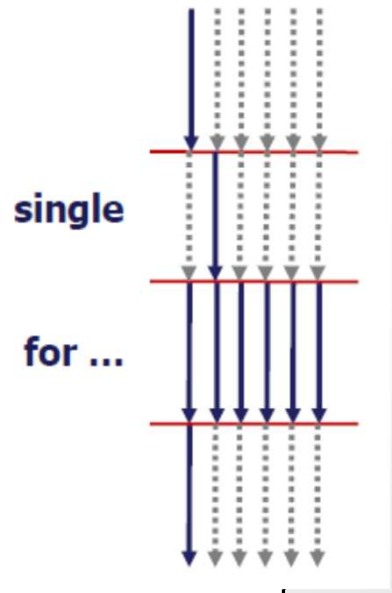
- Provide different sections for parallelization

```
omp_set_num_threads(10);

#pragma omp parallel sections
  {

#pragma omp section
    {
      for(int i=0;i<1000;i++) {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d section A\n", tid);
      }
    }

#pragma omp section
    {
      for(int i=0;i<1000;i++) {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d section B\n", tid);
      }
    }
  }
```

```
Thread ID  0 section B
Thread ID  4 section A
Thread ID  0 section B
Thread ID  4 section A
Thread ID  0 section B
Thread ID  4 section A
Thread ID  0 section B
Thread ID  4 section A
Thread ID  4 section A
Thread ID  4 section A
…
```

SINGLE

- Makes sure that certain block is executed by only one threa




```
#pragma omp parallel
  {
#pragma omp single
    {
      for(int i=0;i<10;i++) {
        int tid = omp_get_thread_num();
        printf("Thread ID %d section A\n", tid);
      }
    } //!!! here's an implicit barrier !!!
#pragma omp for
    for(int i=0;i<100;i++) {
      int tid = omp_get_thread_num();
      printf("Thread ID %d section B\n", tid);
    }
  }
```

```
Thread ID  2 section A
Thread ID  2 section A
Thread ID  2 section A
Thread ID  2 section A
Thread ID  2 section A
Thread ID  2 section A
Thread ID  2 section A
Thread ID  2 section A
Thread ID  2 section A
Thread ID  2 section A
Thread ID  9 section B
Thread ID  9 section B
Thread ID  2 section B
Thread ID  0 section B
Thread ID  7 section B
Thread ID  8 section B
…
```

- *Implicit barrier* at the end of the block

BARRIERS

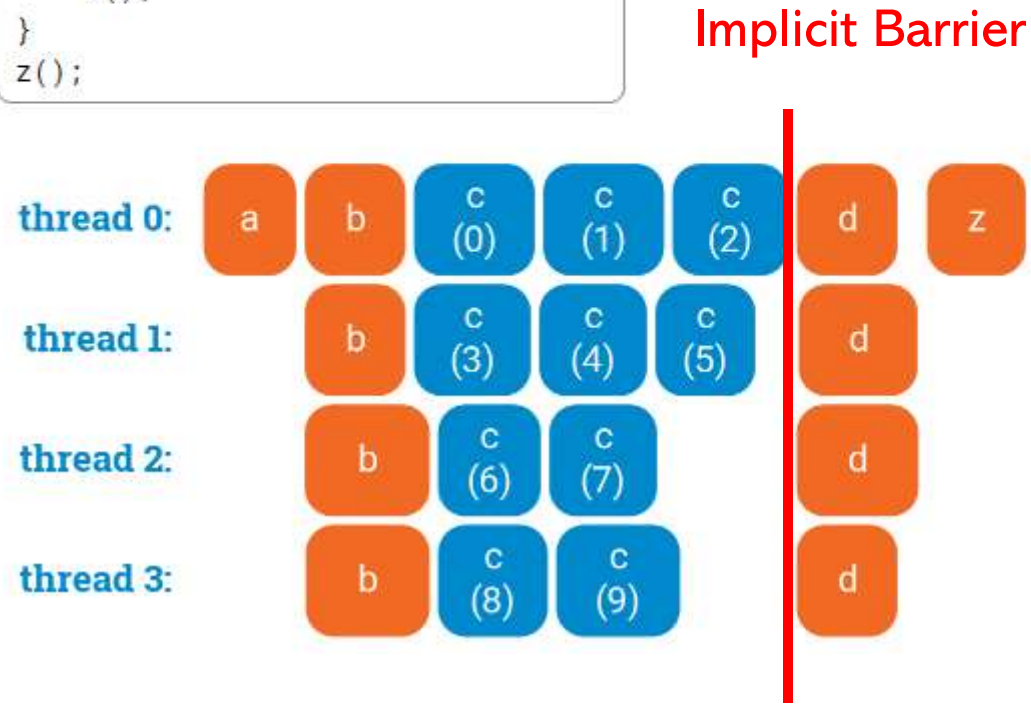
- Wait, did you say, *implicit barrier*?

- OpenMP constructs often has an implied barrier at the end
  - All parallel blocks
  - Single block
  - omp for block
  - (the critical section example shows that it does not have an implicit barrier)

- One of the dirty parts of OpenMP
  - Usually your intuition makes sense
  - Sometimes tricky

IMPLICIT BARRIERS

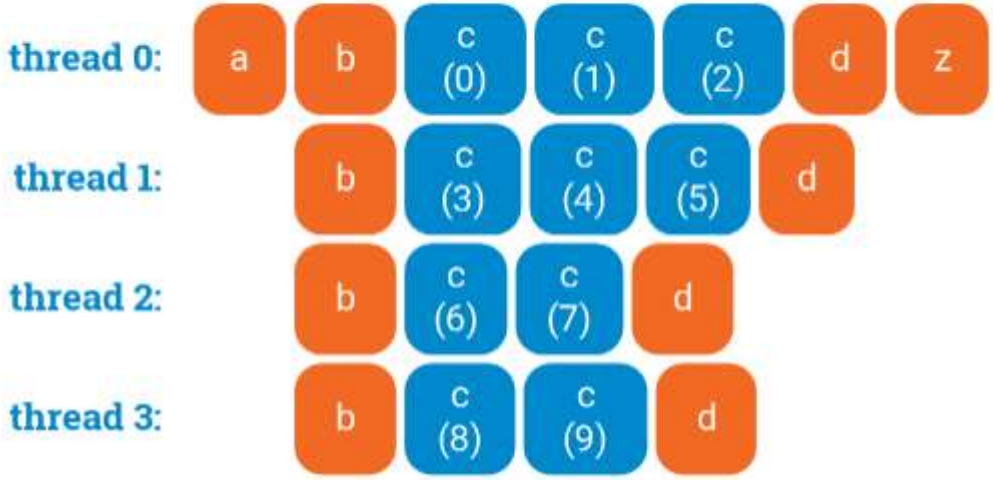
```
a();
#pragma omp parallel
{
    b();
    #pragma omp for
    for (int i = 0; i < 10; ++i) {
        c(i);
    }
    d();
}
z();
```



IMPLICIT BARRIERS

- If you do not want the implicit barrier, use **nowait** clause
  - Removes the implicit barrier

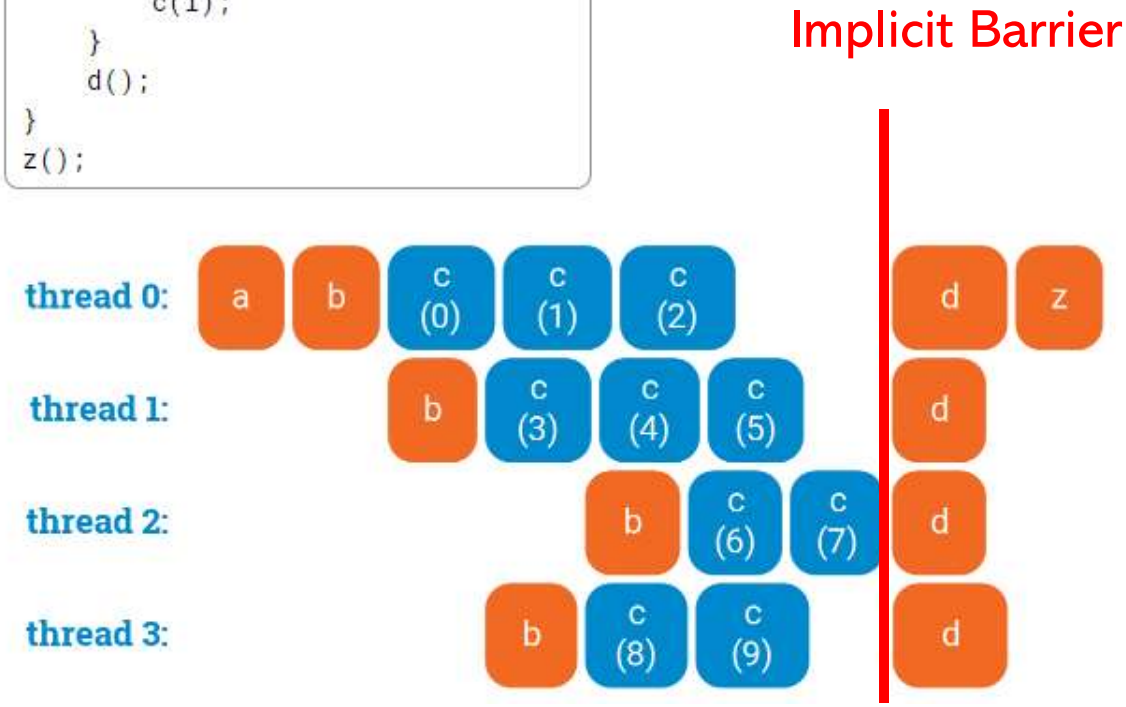
```
a();
#pragma omp parallel
{
    b();
    #pragma omp for nowait
    for (int i = 0; i < 10; ++i) {
        c(i);
    }
    d();
}
z();
```



IMPLICIT BARRIERS

- There is NO synchronization BEFORE the loop

```
a();
#pragma omp parallel
{
    #pragma omp critical
    {
        b();
    }
    #pragma omp for
    for (int i = 0; i < 10; ++i) {
        c(i);
    }
    d();
}
z();
```



EXPLICIT BARRIERS

- If you want a barrier, use
  - `#pragma omp barrier`
  - For all the threads (no setting for the barrier size)

MASTER

- Similar to a section, but ensure that it is executed by the master thread
- *Very different from the *single* clause
  - No implicit barrier

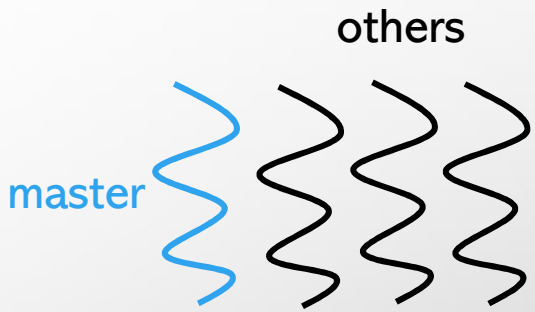
```
#pragma omp parallel
  {
#pragma omp master
    {
      for(int i=0;i<100;i++) {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d section A\n", tid);
      }
    }
#pragma omp for
    for(int i=0;i<100;i++) {
      int tid = omp_get_thread_num();
      printf("Thread ID %2d section B\n", tid);
    }
  }
```

```
Thread ID  2 section B
Thread ID  2 section B
Thread ID  5 section B
Thread ID  5 section B
Thread ID  3 section B
Thread ID  3 section B
Thread ID  3 section B
Thread ID  4 section B
Thread ID  0 section A
Thread ID  4 section B
Thread ID  4 section B
Thread ID  0 section A
Thread ID  0 section A
…
```

MASTER

- Similar to a section, but ensure that it is executed by the master thread
- *Very different from the *single* clause
  - No implicit barrier

```
#pragma omp parallel
  {
#pragma omp master
    {
      for(int i=0;i<100;i++) {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d section A\n", tid);
      }
    }
#pragma omp for
    for(int i=0;i<100;i++) {
      int tid = omp_get_thread_num();
      printf("Thread ID %2d section B\n", tid);
    }
  }
```



MASTER + EXPLICIT BARRIER

- Makes it similar to 'single'

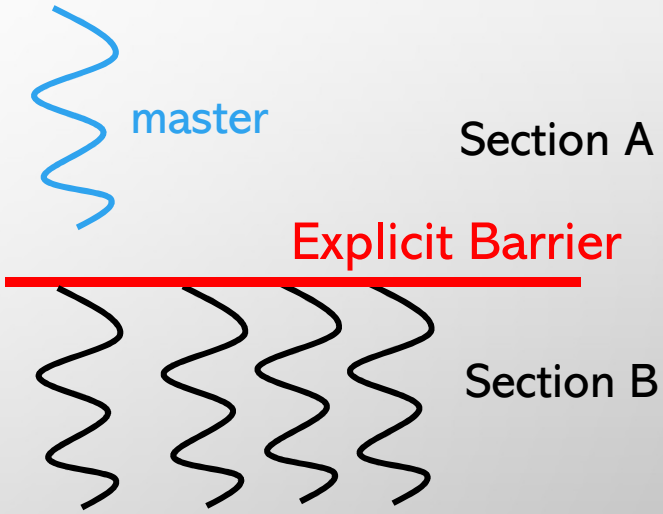
```
#pragma omp parallel
  {
#pragma omp master
    {
      for(int i=0;i<100;i++) {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d section A\n", tid);
      }
    }
#pragma omp barrier
#pragma omp for
    for(int i=0;i<100;i++) {
      int tid = omp_get_thread_num();
      printf("Thread ID %2d section B\n", tid);
    }
  }
```

```
Thread ID  0 section A
Thread ID  0 section A
Thread ID  0 section A
…
Thread ID  0 section A
Thread ID  0 section A
Thread ID  7 section B
Thread ID  7 section B
Thread ID  9 section B
Thread ID  0 section B
Thread ID  0 section B
Thread ID  0 section B
Thread ID  0 section B
Thread ID  5 section B
Thread ID  5 section B
Thread ID  5 section B
Thread ID  1 section B
Thread ID  1 section B
Thread ID  1 section B
…
```

MASTER + EXPLICIT BARRIER

- Makes it similar to 'single'

```
#pragma omp parallel
  {
#pragma omp master
    {
      for(int i=0;i<100;i++) {
        int tid = omp_get_thread_num();
        printf("Thread ID %2d section A\n", tid);
      }
    }
#pragma omp barrier
#pragma omp for
    for(int i=0;i<100;i++) {
      int tid = omp_get_thread_num();
      printf("Thread ID %2d section B\n", tid);
    }
  }
```



REDUCTION

```
int main(int argc, char **argv)
{

  int sum=0;
#pragma omp parallel for
  for(int i=0;i<100;i++) {
    sum+=i;
  }
  printf("Sum: %d\n", sum);
  return 0;
}
```

```
Sum: 1889
```

- Q: how do we get the proper sum?
  - Shared? Private? Lastprivate? Firstprivate?
  - A: None of the above

REDUCTION

```
int main(int argc, char **argv)
{

  int sum=0;
#pragma omp parallel for reduction(+:sum)
  for(int i=0;i<100;i++) {
    sum+=i;
  }
  printf("Sum: %d\n", sum);
  return 0;
}
```

- Reduction clause makes local sums, and add them into the global sum
- We will talk about how to do efficient reduction later

SCHEDULING LOOPS

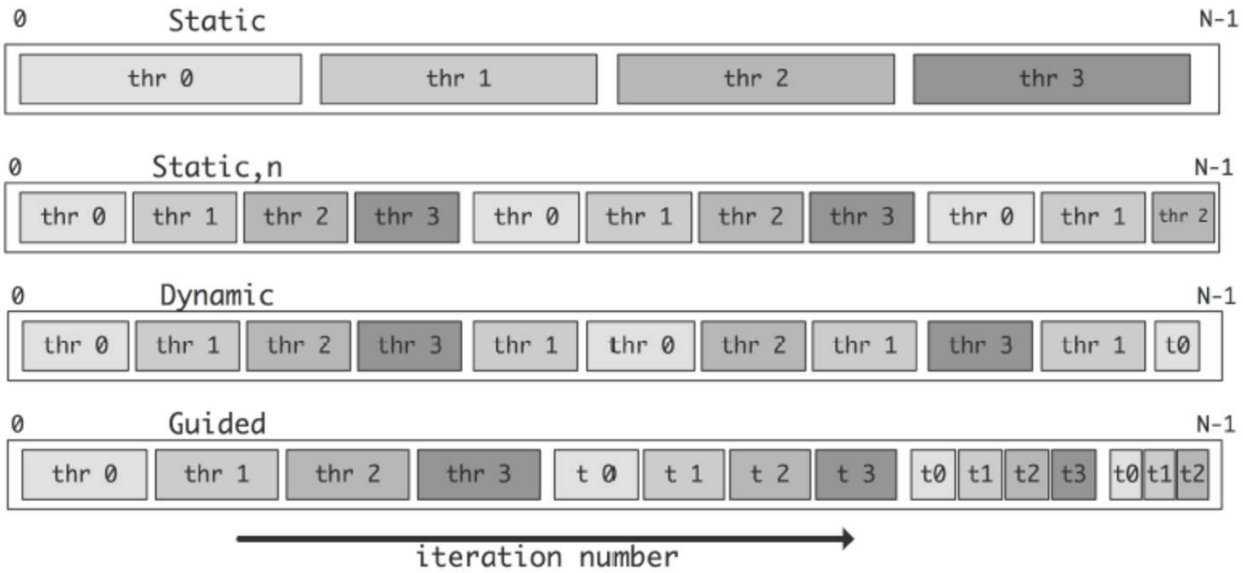
- You can (to some degree) control how the loop is split

```
int main()
{
  …
  int somevar = 0;
#pragma omp parallel for schedule(static, 1024)
  for(long long int i=0;i<N;i++) {
    c[i] += k[i]*a[i];
    c[i] += k[i]*b[i];
    //what happens if somevar accessed here?
  }
  return 0;
}
```

- schedule (static / dynamic / guided, block size(optional))

SCHEDULING LOOPS

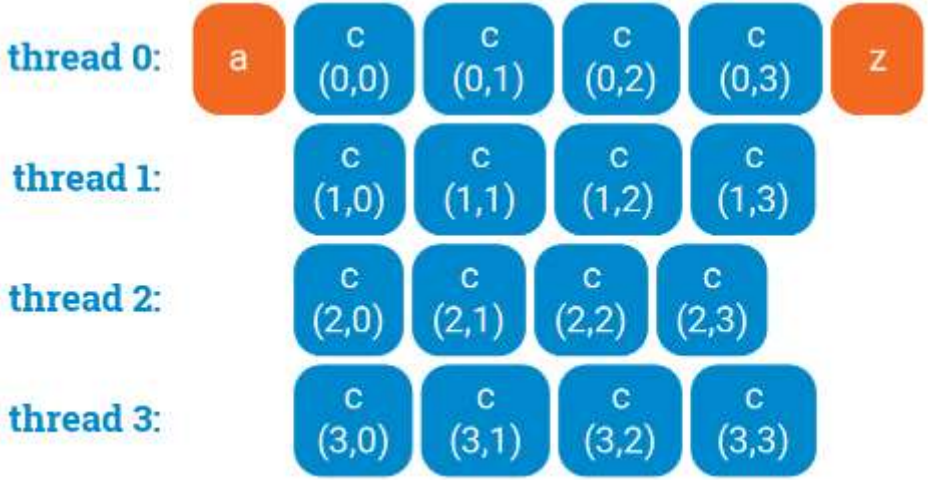
- schedule(static / dynamic / guided, block_size(optional))
  - We have seen this, right?
- Static (default option): iterations are divided into blocks of block_size, and statically assigned to threads
- Dynamic: when a thread finishes a block, it is assigned the next block
  - Useful when work per iteration is irregular
- Guided: a dynamic scheduling, but the block size decreases over time
  - Useful when work per iteration is relatively regular, but threads arrives at varying times



NESTED LOOP

- ‘parallel for’ only parallelizes the outermost loop

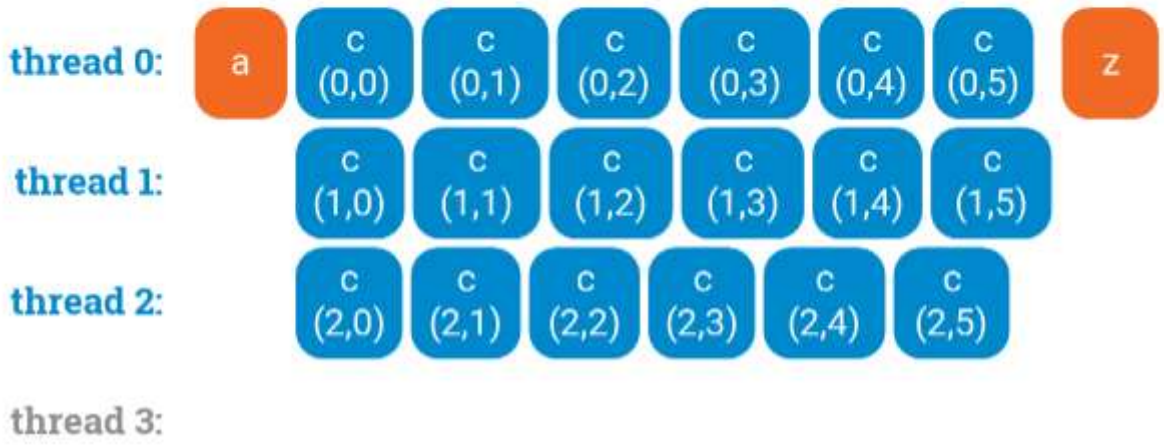
```
a();
#pragma omp parallel for
for (int i = 0; i < 4; ++i) {
    for (int j = 0; j < 4; ++j) {
        c(i, j);
    }
}
z();
```



NESTED LOOP

- What if there are not enough iterations for outermost loop?
  - E.g., machine has 4 threads, but outermost loop has only three iterations

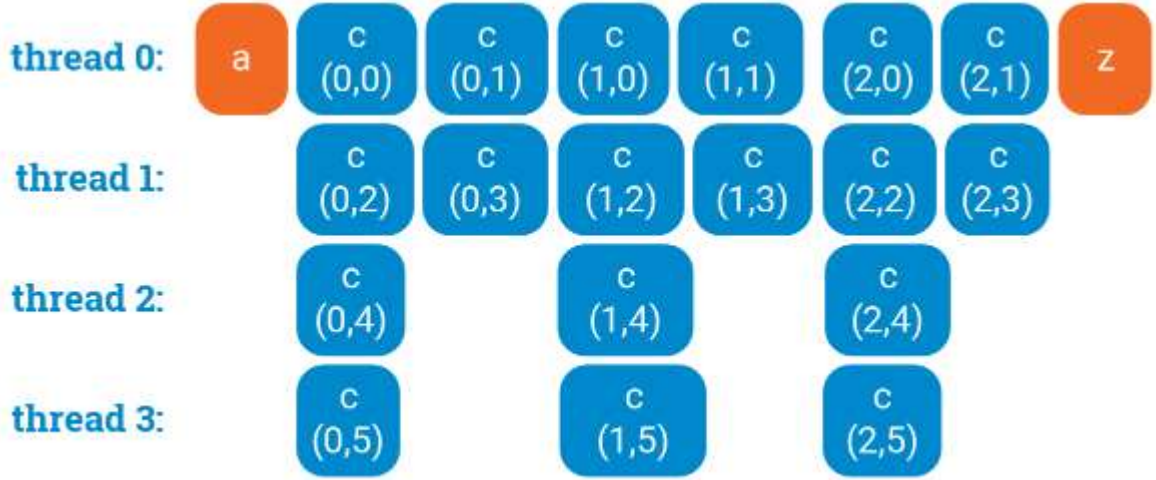
```
a();
#pragma omp parallel for
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 6; ++j) {
        c(i, j);
    }
}
z();
```



BAD WAY (1)

- Parallelizing the inner loop
- Sometimes better
  - But not always
  - In this case, nowait helps

```
a();
for (int i = 0; i < 3; ++i) {
    #pragma omp parallel for
    for (int j = 0; j < 6; ++j) {
        c(i, j);
    }
}
z();
```

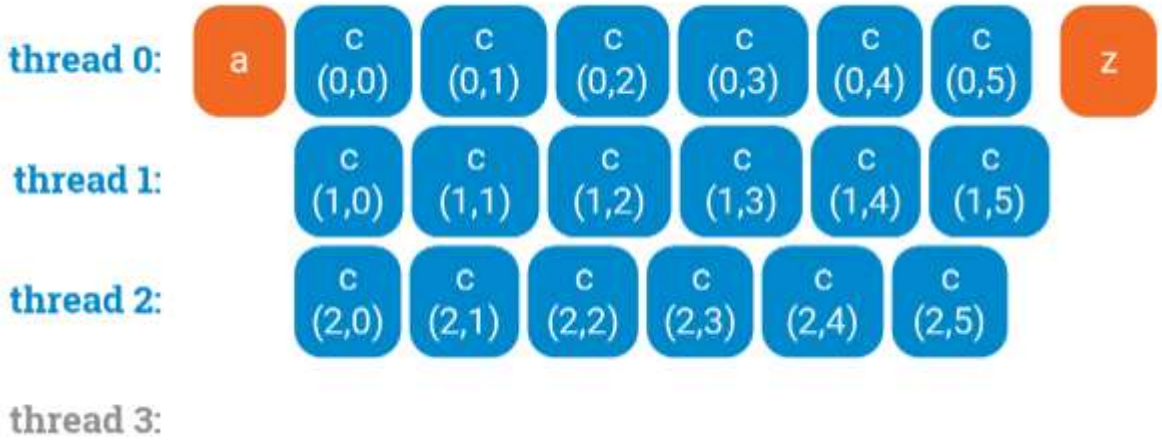


BAD WAY (2)

- Nested parallelism is disabled by default!
- Second pragma ignored

```
a();
#pragma omp parallel for
for (int i = 0; i < 3; ++i) {
    #pragma omp parallel for
    for (int j = 0; j < 6; ++j) {
        c(i, j);
    }
}
z();
```

ignored



BAD WAY (3)

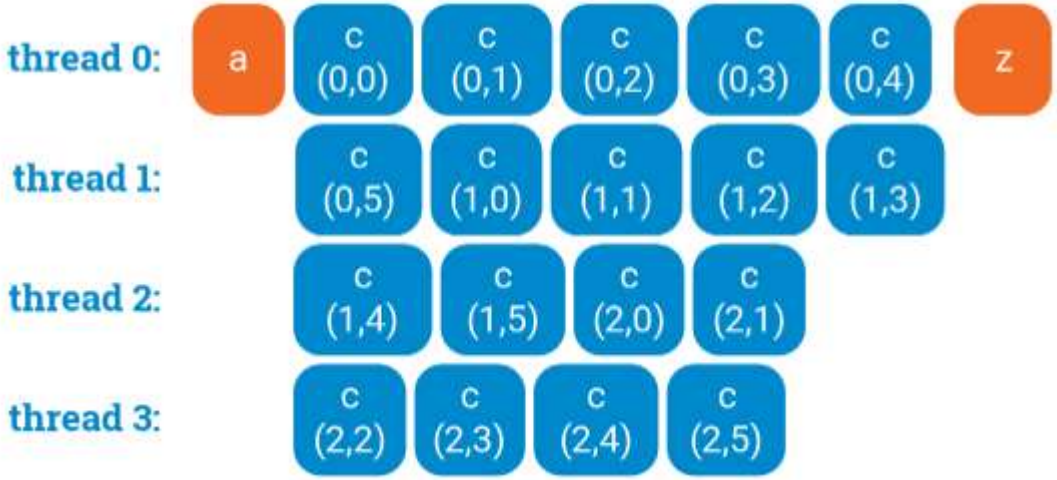
- Nested omp for does not work
- Compile error

```
a();
#pragma omp parallel for
for (int i = 0; i < 3; ++i) {
    #pragma omp for
    for (int j = 0; j < 6; ++j) {
        c(i, j);
    }
}
z();
```

GOOD WAY (1)

- Loop collapse
- 18 iterations in total

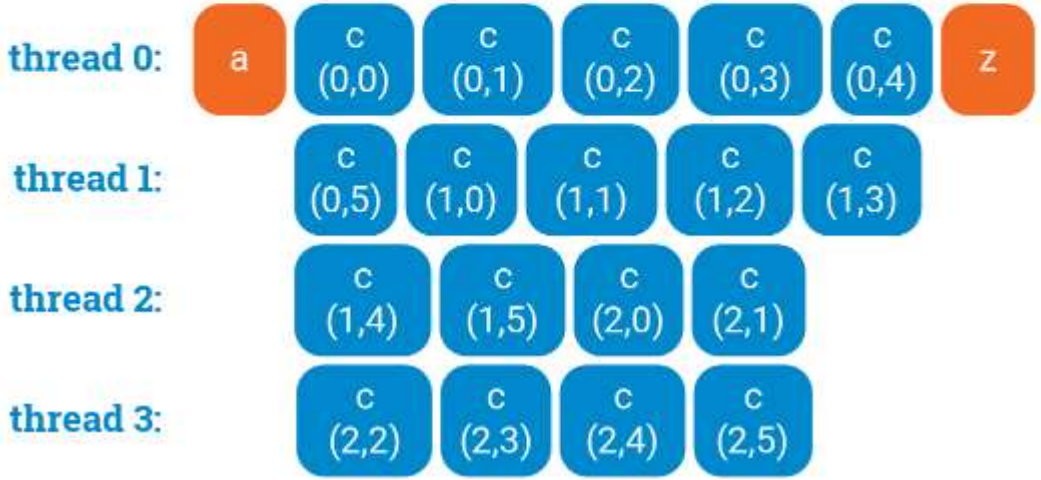
```
a();
#pragma omp parallel for
for (int ij = 0; ij < 3 * 6; ++ij) {
    c(ij / 6, ij % 6);
}
z();
```



GOOD WAY (2)

- Automatic loop collapse
- 18 iterations in total

```
a();
#pragma omp parallel for collapse(2)
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 6; ++j) {
        c(i, j);
    }
}
z();
```



OPENMP

- Super-easy to parallelize simple loop structures
- Does the scheduling for you
- Gets dirty as your jobs become non-uniform
- Hard to figure out when something's wrong

JOB QUEUING?

- So far what we've done can only cover index-based scheduling
- For job queuing, you can look for "OpenMP tasking"
  - A new feature in OpenMP 3.0 (released in 2008)
  - Probably not covered in this lecture,
    but you can use it for assignments if you want

Does a binary tree traversal →

```
void traverse( int n, int size )
{
  if (n >= size) return;
#pragma omp task firstprivate(n)
    traverse( 2*n, size); //left child
#pragma omp task firstprivate(n)
    traverse( 2*n+1, size ); //right child
#pragma omp taskwait //make sure all branches are complete
    usleep(30000);
    printf("Processing %d\n", n);
}
int main()
{
#pragma omp parallel
#pragma omp single
  traverse(1, N);
#pragma omp taskwait
  return 0;
}
```